In the above code, we have done a couple of things. First, we have defined the imports, then we have created the view model, inside which we have the *onItemTap* method bound to the click of the *read more* element. This will get the URL for the post and pass it to the *generateWebRenderingPage* function. This function defines a new page, purely by using the code (and not xml as we have done in *main-view.xml*). This is to demonstrate that if you want to declare the view using code, this is how it can be done.

We have also used the frame object to navigate from one page to another and back to the main page by clicking the *back* button in the second page.

For loading the actual feed from the URL, we have created a module called *RSSFeadLoader*, which has been used in this file. To define this module, create a file called *rss-service.js* and paste the following code into it:

```
var httpModule = require('http');
var parseString = require('nativescript-xml2js').parseString;

function RSSFeadLoader() {
    var feeds = [];
    this.loadData = function(context) {

        httpModule.getString("https://opensourceforu.com/
feed").then((r) =>
{
            parseString(r, function(err, result) {
                feeds = result.rss.channel[0].item;
                context.feeds = feeds;
                return feeds;
            });
        }, (e) => {
            console.log("Error loading RSS Feed");
        });
    }
}

module.exports = RSSFeadLoader;
```

The above code will load the RSS feed from the *opensourceforu.com* website and bind it to the feed property of the model.

Finally, we have to bind the model's context with our view that we have declared in the *main-view.xml* file.

We will do this in the *main-page.js* file with the following lines of code:

```
var MainViewModel = require("./main-view-model");
var mainViewModel = new MainViewModel();

function pageLoaded(args) {
    var page = args.object;
    page.bindingContext = mainViewModel;
}
exports.pageLoaded = pageLoaded;
```

Now head to *app-root.xml* file and set your default page to *main-view.xml* by setting the *defaultPage* property. The property should be set as *"defaultPage=" main-page"*. This is pretty much all you need for the application. You can now test it using the *tns run android* or *tns run ios* command.

The entire code for the application (with better comments) can be found at *github.com/shivasaxena/RSSReaderMobileApp*, so you can set up the development environment and clone the code from GitHub to try it out.

Tools to try out

- ***Building the same code for a different platform:*** In case you are using the Windows platform and want to build for iOS or need support, you can use NativeScript Sidekick. It allows you to have 100 builds/month/user for free. You can check it out at NativeScript Sidekick (*https://www.nativescript.org/nativescript-sidekick*).
- ***Quickly trying out NativeScript:*** If you want to quickly try out NativeScript without any local setup, you can head to *https://play.nativescript.org*.
- ***NativeScript documentation:*** To further explore NativeScript, the official documentation can be found at *https://docs.nativescript.org*. END

By: Shiva Saxena

The author is a FOSS enthusiast, currently working on IoT and cloud technologies at SAP. He can be reached at *shivasaxena@outlook.com*. Thoughts expressed in the article belong solely to the author and do not represent SAP.